

NextTrackAssist

pytest 完全学習ガイド

テストの基礎から実装コードの完全解読まで

— `conftest.py` / フィクスチャ / 5つのテストファイル完全解説 —

全10章 + チートシート

目次

Chapter 1 テストとは何か

- 1-1 手動テストの問題
- 1-2 自動テストが解決すること
- 1-3 このプロジェクトのテスト全体像

Chapter 2 pytest の基本ルール

- 2-1 ファイル・関数の命名ルール
- 2-2 assert の使い方
- 2-3 pytest の実行方法

Chapter 3 フィクスチャとは何か

- 3-1 なぜフィクスチャが必要か
- 3-2 フィクスチャの基本構文
- 3-3 フィクスチャの依存関係

Chapter 4 conftest.py 完全解説

- 4-1 環境変数の書き換え
- 4-2 reset_database フィクスチャ (autouse)
- 4-3 app / client フィクスチャ
- 4-4 user_with_tracks フィクスチャ
- 4-5 logged_in_client フィクスチャ

Chapter 5 test_app.py — 結合テスト

- 5-1 SECRET_KEY 読み込み確認
- 5-2 トラックのスコープ確認
- 5-3 他ユーザーのトラック保護

Chapter 6 test_auth.py — 認証テスト

- 6-1 登録 (正常・重複・短パスワード)
- 6-2 ログイン (成功・失敗)
- 6-3 ログアウト

Chapter 7 test_tracks.py — CRUD テスト

- 7-1 認証ガード
- 7-2 トラック登録バリデーション
- 7-3 削除と認可

Chapter 8 test_api.py — REST API テスト

- 8-1 未ログインのリダイレクト
- 8-2 トラック一覧 JSON
- 8-3 推薦 API レスポンス

Chapter 9 test_score.py — 単体テスト

9-1 単体テストとは

9-2 各スコア関数のテスト

9-3 BPM 除外ロジック

Chapter 10 テストの書き方（実践）

10-1 テストを書く4ステップ

10-2 新しいテストを書いてみる

10-3 チートシート

Chapter 1

テストとは何か

コードの動作を自動で確認する仕組みを基礎から理解する

1-1 手動テストの問題

今まで機能を確認するとき、こういう手順を踏んでいなかっただろうか？

1	コードを書く
2	ブラウザを開いて手でフォームに入力
3	ボタンを押して動作確認
4	別の機能を確認する…
5	最初に確認したところが壊れてないか不安になる
6	また手で確認…（繰り返し）

⚠️ これがしんどい理由：毎回時間がかかる / 人間は全パターン確認できない / コードを変えるのが怖くなる

1-2 自動テストが解決すること

「ブラウザで手動確認する作業をコードにしたもの」が自動テスト。一度書いたら何度でも一瞬で実行できる。

手動 → コードへの変換イメージ

手動でやっていたこと：

ブラウザで /register にアクセス → フォームに入力 → 送信 → /login にリダイレクトされたか確認

これをコードにすると：

```
# test_auth.py ■■  
  
def test_register_success(client):  
    response = client.post('/register', data={  
        'email': 'new@example.com',
```

```
'password': 'securepass'

})

assert response.status_code == 302 # ■■■■■■■■■■

assert '/login' in response.headers['Location'] # /login ■
```

テストを書くメリット：コード変更後に「壊れた場所」を一瞬で発見できる。
リファクタリングや機能追加が怖くなる。

1-3 このプロジェクトのテスト全体像

NextTrackAssist には tests/ フォルダに6つのファイルがある。

ファイル	役割	テスト種別
conftest.py	フィクスチャ（共通部品）の定義	設定
test_app.py	アプリ全体の動作確認	結合テスト
test_auth.py	登録・ログイン・ログアウト	結合テスト
test_tracks.py	トラック CRUD・バリデーション	結合テスト
test_api.py	REST API の JSON レスポンス確認	結合テスト
test_score.py	スコア計算ロジックの単体確認	単体テスト


```

assert 'Track A' in body # HTML
assert user is not None # DB
assert user is None # DB
assert len(data) == 3 # JSON
assert result['total_score'] == 100.0 #

```

2-3 pytest の実行方法

```

#
#
pytest

#
pytest tests/test_auth.py

#
pytest tests/test_auth.py::test_register_success

#
pytest -v # 1 verbose
pytest --tb=short #
pytest -v --tb=short #

```

CI (GitHub Actions) では以下のコマンドが使われている：

```

# ci.yml
pytest --tb=short -v

```


pytest が引数の名前を見て「このフィクスチャが必要だな」と自動判断し、実行前に準備してくれる。明示的に呼び出す必要はない。

3-3 yield を使った前処理・後処理

yield を使うと「テスト前の処理」と「テスト後の処理」を両方書ける。

```
# yield ████

@pytest.fixture()

def db_session():

    # █ ████ ████████████████████

    db = create_session()

    db.create_all() # ████████

    yield db # ← ████████████████

    # █ ████ ████████████████████

    db.drop_all() # ████████████████████

    db.close()
```

NextTrackAssist の reset_database フォックスチャがまさにこのパターン。テストごとにDBをリセットして「クリーンな状態」から始められる。

3-4 フィクスチャの依存関係

```
# ██████████

@pytest.fixture()

def app():

    return create_app() # Flask██████

@pytest.fixture()

def client(app): # ← app ██████████

    return app.test_client() # app ██████

def test_index(client): # ← client ██████████

    # pytest ████ app → client ██████████

    response = client.get('/')

    assert response.status_code == 302
```

フィクスチャは他のフィクスチャを引数に取ることができる。pytest が依存関係を自動解決して正しい順番で準備してくれる。

conftest.py 完全解説

4-1 環境変数の書き換え (import より前！)

なぜ import より前に書くのか？

sqlite:// の意味

NextTrackAssist pytest 完全学習ガイド — 9 —

メモリ SQLite を使うことで：本番 DB を汚さない / テスト終了後に自動で消える / PostgreSQL より高速に動く

4-2 reset_database フィクスチャ (autouse)

```
# conftest.py

@pytest.fixture(autouse=True) # autouse=True = ■■■■■■■■■■

def reset_database():

    Base.metadata.drop_all(bind=engine) # ■■■■■■■■

    Base.metadata.create_all(bind=engine) # ■■■■■■■■■■

    yield # ← ■■■■■■■■■■■■■■■■

    Base.metadata.drop_all(bind=engine) # ■■■■■■■■
```

autouse=True の挙動：

```
# ■■■■■■

test_1 ■■■■

■■ reset_database: drop → create■■■■■■■■DB■

■■ yield → test_1 ■■■■■■

■■ reset_database: drop■■■■■■■

test_2 ■■■■

■■ reset_database: drop → create■■■■■■■■DB■■■■■

■■ yield → test_2 ■■■■■■

■■ reset_database: drop
```

⚠ これがないと test_1 で作ったデータが test_2 に残り、テストの順番によって結果が変わる「不安定なテスト」になってしまう。

4-3 app / client フィクスチャ

```
# conftest.py

@pytest.fixture()

def app():

    return create_app() # Flask ■■■■■■■■■■■■■■■■

@pytest.fixture()

def client(app): # app ■■■■■■■■■■

    return app.test_client() # ■■■■ HTTP ■■■■■■
```

`app.test_client()` は Flask が提供するテスト用クライアント。実際のサーバーを起動せずに HTTP リクエストを送れる。

4-4 user_with_tracks フィクスチャ

[illegible]

Hidden Track (other のトラック) が重要。「自分のトラックしか見えない」「他人のトラックは操作できない」という認可テストで使われる。

4-5 logged_in_client フィクスチャ

[illegible]

```
# ■■■  
  
def test_something(logged_in_client):  
    client, user = logged_in_client # ■■■■■■  
  
    # ■■■■ client ■■■■■■■■■■  
  
    response = client.get('/') # ■■■■■■■■■■
```

client

がセッション（Cookie）を保持するため、ログイン後の全リクエストが認証済みの状態で処理される。

Chapter 5

test_app.py — 結合テスト

アプリ全体の「繋がり」を確認するテスト

5-1 SECRET_KEY の読み込み確認

```
# test_app.py

def test_create_app_loads_secret_key(app):
    # conftest.py ■■■■■■ Flask ■■■■■■■■■■■■

    assert app.config['SECRET_KEY'] == 'test-secret-key'
```

最もシンプルなテスト。conftest.py で設定した環境変数が config.py → Flask アプリまで正しく伝わっているかを確認する。

5-2 トラックのスコープ確認（重要）

```
# test_app.py

def test_tracks_are_scoped_to_logged_in_user(client, user_with_tracks):
    # Step 1: ■■■■
    login(client) # ← ■■■■■■■■■■

    # Step 2: ■■■■■■■■■■
    response = client.get('/')
    body = response.get_data(as_text=True) # HTML ■■■■■■■■■■

    # Step 3: ■■
    assert response.status_code == 200
    assert 'Track A' in body # ■ ■■■■■■■■■■
    assert 'Track B' in body # ■ ■■■■■■■■■■
    assert 'Hidden Track' not in body # ■ ■■■■■■■■■■
```

⚠ このテストが「認可（Authorization）」の確認。ログインしていれば誰のデータでも見えてしまう実装ミスを防ぐ。

5-3 他ユーザーのトラック保護

```
# test_app.py
```

```
def test_cannot_open_other_users_tracks(client, user_with_tracks):
    login(client)

    # Hidden Track■■■■■■■■■■ID■ DB ■■■■

    db = SessionLocal()

    hidden_track = db.query(Track).filter(Track.title == 'Hidden Track').first()

    db.close()

    # ■■■ID■■■■■■■■■■

    response = client.get(f'/tracks/{hidden_track.id}')

    assert response.status_code == 404 # 404 ■■■■
```

URL を直接叩いてアクセスしようとしても 404 になることを確認。これは `routes/tracks.py` の `_find_user_track()` 関数が `user_id` でフィルタしているから。

@login_required デコレーターが正しく機能しているかを確認する。未ログイン状態でアクセスすると /login にリダイレクトされるはず。

7-2 トラック登録テスト

NextTrackAssist pytest 完全学習ガイド — 17 —

```
assert track.key == '8A'
```

7-3 バリデーションテスト

```
# test_tracks.py

def test_create_track_bpm_too_low(logged_in_client):
    client, _ = logged_in_client # user ████████ _ ████

    response = client.post('/tracks', data={
        'title': 'T', 'artist': 'A',
        'bpm': '10', # ← 40 ██████
        'key': '8A', 'energy': '6'
    }, follow_redirects=True)

    body = response.get_data(as_text=True)
    assert '40' in body or '■' in body # ██████████

def test_create_track_invalid_key_format(logged_in_client):
    client, _ = logged_in_client

    response = client.post('/tracks', data={
        'title': 'T', 'artist': 'A',
        'bpm': '128',
        'key': 'ZZ', # ← Camelot ████████
        'energy': '6'
    }, follow_redirects=True)

    body = response.get_data(as_text=True)
    assert 'Camelot' in body or 'Key' in body
```

7-4 削除と認可テスト

```
# test_tracks.py

def test_delete_track_success(logged_in_client):
    client, user = logged_in_client

    # ████████ ID ████

    db = SessionLocal()
    track = db.query(Track).filter(Track.user_id == user.id).first()
    track_id = track.id
    db.close()
```


test_api.py — REST API テスト

8-1 テスト用ヘルパー関数

[illegible]

フィクスチャ（`conftest.py`）と違い、`test_api.py` 専用のヘルパー関数として定義されている。_から始まる命名は「内部用・外部から直接呼ばない」の慣例。

8-2 トラック一覧 API テスト

```
# test_api.py

def test_unauthenticated_redirects(client):
    """■■■■■■ /api/tracks ■■■■■■ 302 ■■■■■"""
    resp = client.get('/api/tracks')
    assert resp.status_code == 302 # JSON ■■■■■■

def test_list_tracks_returns_user_tracks(client):
    """■■■■■■■■■■■■■■■■■■■■ JSON ■■■■■"""
    user_id = _create_user_and_login(client)
    _add_tracks(user_id, count=3)

    resp = client.get('/api/tracks')
    assert resp.status_code == 200

    data = resp.get_json() # JSON ■■■■■■
    assert len(data) == 3 # 3■■■■■
    assert 'title' in data[0] # ■■■■■
    assert 'bpm' in data[0]
```

`resp.get_json()` は `Content-Type: application/json` のレスポンスを自動でパースして Python の `dict` / `list` に変換してくれる。

8-3 推薦 API テスト

```
# test_api.py

def test_recommend_success(client):
    """■■ API ■■■■■■ JSON ■■■■■"""
    user_id = _create_user_and_login(client)
    track_ids = _add_tracks(user_id, count=3)

    resp = client.get(f'/api/recommend/{track_ids[0]}')
    assert resp.status_code == 200

    data = resp.get_json()
    assert 'base_track' in data # ■■■■■■■■■■
    assert 'recommendations' in data
```

test_score.py — 単体テスト

Flask を使わず関数を直接呼ぶ、最もシンプルなテスト

9-1 単体テストとは

Chapter 5～8 は「Flask を経由した結合テスト」だったが、test_score.py だけは Flask も DB も使わない純粋な単体テスト。

	結合テスト (test_auth, test_tracks...)	単体テスト (test_score)
対象	HTTP リクエスト → DB → レスポンス	関数の入出力だけ
速さ	少し遅い (DB接続あり)	非常に高速
準備	client, user_with_tracks... 必要	不要 (import だけ)
目的	機能全体の動作確認	ロジックの正確さ確認

9-2 各スコア関数のテスト

```
# test_score.py

from app.services.score import (
    calc_bpm_score,
    calc_energy_score,
    calc_key_score,
    calc_total_score,
)

def test_bpm_exact():
    """BPM■■■0 → ■■■100"""
    score, diff = calc_bpm_score(128, 128) # ■■■■■■
    assert score == 100
    assert diff == 0

def test_key_exact():
    """Key ■■■■■ → ■■■100"""
    assert calc_key_score('8A', '8A') == 100
```

```
def test_energy_exact():
    """Energy■■0 → ■■■100"""
    assert calc_energy_score(6, 6) == 100

def test_total_score_perfect_match():
    """BPM■Key■Energy ■■■■■■ → ■■■■■100"""
    base = {'bpm': 128, 'key': '8A', 'energy': 6}
    result = calc_total_score(base, base) # ■■■■■■
    assert result['total_score'] == 100.0
```

9-3 BPM 除外ロジックのテスト

```
# test_score.py

def test_total_score_excluded_when_bpm_diff_too_large():
    """BPM■■■6■■■■■■■ None ■■■■■■■■■■■■■■"""
    base = {'bpm': 128, 'key': '8A', 'energy': 6}
    far = {'bpm': 142, 'key': '8A', 'energy': 6} # ■14
    assert calc_total_score(base, far) is None
```

score.py の核心ロジック「BPM差が6を超えたら推薦から除外する」という動作を直接テストしている。このロジックが壊れると recommend や generate_set 全体がおかしくなる。

テストの書き方 (実践)

10-1 テストを書く 4 ステップ

Step	問い	例
1	何をテストするか？	rekordbox CSV をインポートしたらDBに保存される
2	準備は何が必要か？	ログイン済みのクライアント / CSV ファイル（バイト列）
3	どんなリクエストを送るか？	POST /tracks/import に csv_file つきで送る
4	何を確認するか？	302でリダイレクト / DBにトラックが保存されている

4ステップの考え方でコードにしていくと：

NextTrackAssist pytest 完全学習ガイド — 24 —

チートシート

フィクスチャー一覧 (conftest.py)

フィクスチャ名	提供するもの	autouse
reset_database	全テスト前後にDBをリセット	全テスト
app	Flask アプリインスタンス	—
client	テスト用 HTTP クライアント	—
user_with_tracks	ユーザー2人 + トラック3件	—
logged_in_client	ログイン済みクライアント + ユーザー	—

よく使う assert パターン

```
# ■■■■■■■■

# HTTP ■■■■■■■■

assert response.status_code == 200 # OK
assert response.status_code == 302 # ■■■■■■■■
assert response.status_code == 404 # Not Found

# ■■■■■■■■

assert '/login' in response.headers['Location']

# HTML ■■

body = response.get_data(as_text=True)
assert 'Track A' in body # ■■■■■■■■■■
assert 'Hidden Track' not in body # ■■■■■■■■■■

# DB ■■■■

record = db.query(Model).filter(...).first()
assert record is not None # ■■■■
assert record is None # ■■■■■■
assert record.bpm == 128 # ■■■■

# JSON ■■■■■■■■API ■■■■

data = response.get_json()
assert len(data) == 3 # ■■
assert 'title' in data[0] # ■■■■■■■■
```

テスト実行コマンド

```
# ■■■■  
  
pytest # ■■■■  
  
pytest -v # ■■■■■■  
  
pytest --tb=short # ■■■■■■  
  
pytest tests/test_auth.py # ■■■■■■  
  
pytest tests/test_auth.py::test_register_success # ■■■■  
  
pytest -v --tb=short # ■■■■■■
```

テストを書く 4 ステップ（再掲）

Step 1	何をテストするか？「～したとき～になること」を1文で言えるか
Step 2	準備は何が必要か？ DB? ログイン済みユーザー? CSVファイル?
Step 3	どんなリクエストを送るか？ GET? POST? どのエンドポイント?
Step 4	何を確認するか？ ステータスコード / HTML内容 / DBの状態

NextTrackAssist pytest 完全学習ガイド 以上